

Window Function Kata

Maya Shapira, Dror Leibov

exercise 1:

Create sales data and build a window per category of sales. Use functions to get the number of items in the window.

```
WITH Sales AS (  
    SELECT 'Computer X' AS product, 'Electronics' AS category, 2026 AS `year`, 100  
    AS volume, 1000 AS price  
    UNION ALL  
    SELECT 'Phone', 'Electronics', 2022, 50, 800  
    UNION ALL  
    SELECT 'Chair', 'Furniture', 2025, 30, 200  
    UNION ALL  
    SELECT 'Table', 'Furniture', 2020, 40, 500  
)  
  
SELECT *,  
    COUNT(product) OVER (  
        PARTITION BY category  
    ) AS num_of_products  
FROM Sales;
```

exercise 2:

Add to the previous question the two first items ordered by price.

```
WITH Sales AS (  
    SELECT 'Computer X' AS product, 'Electronics' AS category, 2026 AS year, 100  
    AS volume, 1000 AS price  
    UNION ALL SELECT 'Phone A', 'Electronics', 2026, 250, 700  
    UNION ALL SELECT 'Chair A', 'Furniture', 2026, 300, 200  
    UNION ALL SELECT 'Sofa B', 'Furniture', 2026, 50, 1200  
)  
SELECT  
    product,  
    category,  
    year,  
    price,  
    COUNT(*) OVER (PARTITION BY category) AS items_in_window,  
    FIRST_VALUE(product) OVER (  
        PARTITION BY category  
        ORDER BY price  
    ) AS first_item,  
    NTH_VALUE(product, 2) OVER (  
        PARTITION BY category
```

```

ORDER BY price
) AS second_item
FROM Sales;
ROWS BETWEEN UNBOUNDED
PRECEDING AND UNBOUNDED FOLLOWING.

```

exercise 3.a:

Compute per product the average price.

```

WITH Sales AS (
  SELECT 'Computer X' AS product, 'Electronics' AS category, 2025 AS year, 110
  AS volume, 1050 AS price
  UNION ALL SELECT 'Computer X', 'Electronics', 2026, 100, 1000
  UNION ALL SELECT 'Phone A', 'Electronics', 2025, 240, 650
  UNION ALL SELECT 'Phone A', 'Electronics', 2026, 250, 700
)
SELECT
  product,
  category,
  year,
  price,
  AVG(price) OVER (PARTITION BY product) AS avg_product_price
FROM Sales;

```

exercise 3.b:

Add to the previous question the product average price over the category average price.

```

WITH Sales AS (
  SELECT 'Computer X' AS product, 'Electronics' AS category, 2025 AS year, 110
  AS volume, 1050 AS price
  UNION ALL SELECT 'Computer X', 'Electronics', 2026, 100, 1000
  UNION ALL SELECT 'Phone A', 'Electronics', 2025, 240, 650
  UNION ALL SELECT 'Phone A', 'Electronics', 2026, 250, 700
)
SELECT
  product,
  category,
  year,
  price,
  AVG(price) OVER (PARTITION BY product) AS avg_product_price,

```

```

    AVG(price) OVER (PARTITION BY category) AS avg_category_price,
    AVG(price) OVER (PARTITION BY product)
    / AVG(price) OVER (PARTITION BY category) AS product_over_category
FROM Sales;

```

exercise 4:

Add to the previous question the product average price over the year average price.

How many windows did you use? How did you name them and why?

```

WITH Sales AS (
    SELECT 'Computer X' AS product, 'Electronics' AS category, 2025 AS year, 110
    AS volume, 1050 AS price
    UNION ALL SELECT 'Computer X', 'Electronics', 2026, 100, 1000
    UNION ALL SELECT 'Phone A', 'Electronics', 2025, 240, 650
    UNION ALL SELECT 'Phone A', 'Electronics', 2026, 250, 700
    UNION ALL SELECT 'Chair A', 'Furniture', 2025, 160, 190
    UNION ALL SELECT 'Chair A', 'Furniture', 2026, 300, 200
)
SELECT
    product,
    category,
    year,
    price,
    AVG(price) OVER product_window AS avg_product_price,
    AVG(price) OVER category_window AS avg_category_price,
    AVG(price) OVER year_window AS avg_year_price,
    AVG(price) OVER product_window
    / AVG(price) OVER category_window AS product_over_category,
    AVG(price) OVER product_window
    / AVG(price) OVER year_window AS product_over_year
FROM Sales
WINDOW
    product_window AS (PARTITION BY product),
    category_window AS (PARTITION BY category),
    year_window AS (PARTITION BY year);

```

Answer: 3 windows were used. product_window is for calculations per product, category_window is for calculations per category, and year_window is for calculations per year.

exercise 5:

Compute per product its price in the last 5 years.

Add a product that had less years of sales.

```

WITH Sales AS (
  SELECT 'Phone' AS product, 'Electronics' AS category, 2018 AS `year`, 20 AS
volume, 800 AS price
  UNION ALL
  SELECT 'Phone', 'Electronics', 2019, 50, 700
  UNION ALL
  SELECT 'Phone', 'Electronics', 2020, 45, 650
  UNION ALL
  SELECT 'Phone', 'Electronics', 2021, 50, 700
  UNION ALL
  SELECT 'Phone', 'Electronics', 2022, 60, 1000
  UNION ALL
  SELECT 'Phone', 'Electronics', 2023, 65, 1100
  UNION ALL
  SELECT 'Chair', 'Furniture', 2023, 40, 230
  UNION ALL
  SELECT 'Chair', 'Furniture', 2024, 15, 180
)

SELECT *,
  AVG(price) OVER (
    PARTITION BY product
    ORDER BY `year`
    ROWS BETWEEN 4 PRECEDING AND CURRENT ROW
  ) AS avg_price_last_5_years
FROM Sales;

```

exercise 6:

Find all the cases in which a product price is lower than its price the year before.

```

WITH Sales AS (
  SELECT 'Phone' AS product, 'Electronics' AS category, 2019 AS `year`, 20 AS
volume, 800 AS price
  UNION ALL
  SELECT 'Phone', 'Electronics', 2020, 50, 700
  UNION ALL
  SELECT 'Phone', 'Electronics', 2021, 45, 900
  UNION ALL
  SELECT 'Computer X', 'Electronics', 2019, 50, 700
  UNION ALL
  SELECT 'Computer X', 'Electronics', 2020, 60, 1000
  UNION ALL
  SELECT 'Chair', 'Furniture', 2020, 40, 230
  UNION ALL
  SELECT 'Chair', 'Furniture', 2021, 15, 180
)

SELECT *

```

```

FROM (
    SELECT *,
        LAG(price) OVER (
            PARTITION BY product
            ORDER BY `year`
        ) AS previous_year_price
    FROM Sales
) AS Sales_with_previous_price
WHERE price < previous_year_price;

```

exercise 7:

In the year of 2026, per category, rank the product by total revenue (price*volume).

```

WITH Sales AS (
    SELECT 'Phone' AS product, 'Electronics' AS category, 2026 AS `year`, 20 AS
    volume, 800 AS price
    UNION ALL
    SELECT 'Computer X', 'Electronics', 2026, 50, 1000
    UNION ALL
    SELECT 'Computer Y', 'Electronics', 2026, 45, 900
    UNION ALL
    SELECT 'Chair', 'Furniture', 2026, 40, 230
    UNION ALL
    SELECT 'Table', 'Furniture', 2026, 15, 180
    UNION ALL
    SELECT 'Couch', 'Furniture', 2026, 30, 780
),

Sales_revenue AS (
    SELECT *,
        price * volume AS revenue
    FROM Sales
)

SELECT *,
    RANK() OVER (
        PARTITION BY category
        ORDER BY revenue DESC
    ) AS revenue_rank
FROM Sales_revenue;

```

exercise 8:

In the year of 2026, create an example with a difference between rank and dense_rank.

```

WITH Sales AS (
    SELECT 'Computer X' AS product, 'Electronics' AS category, 2026 AS year, 100
    AS volume, 1000 AS price
    UNION ALL SELECT 'Phone A', 'Electronics', 2026, 250, 700
    UNION ALL SELECT 'Tablet B', 'Electronics', 2026, 250, 700
    UNION ALL SELECT 'Headphones C', 'Electronics', 2026, 300, 200
)
SELECT
    product,
    category,
    total_revenue,
    RANK() OVER (
        PARTITION BY category
        ORDER BY total_revenue DESC
    ) AS rank_value,
    DENSE_RANK() OVER (
        PARTITION BY category
        ORDER BY total_revenue DESC
    ) AS dense_rank_value
FROM (
    SELECT
        product,
        category,
        SUM(price * volume) AS total_revenue
    FROM Sales
    WHERE year = 2026
    GROUP BY product, category
) AS product_revenue;

```

Phone A and Tablet B have the same revenue: $700 * 250 = 175000$. Therefore RANK produces 1,1,3,... while DENSE_RANK produces 1,1,2,...

exercise 9:

Per category, rank the product by sum of revenue (price*volume) in all years.

```

WITH Sales AS (
    SELECT 'Phone' AS product, 'Electronics' AS category, 2024 AS `year`, 20 AS
    volume, 800 AS price
    UNION ALL
    SELECT 'Phone', 'Electronics', 2025, 40, 400
    UNION ALL
    SELECT 'Phone', 'Electronics', 2026, 50, 1000
    UNION ALL
    SELECT 'Computer X', 'Electronics', 2024, 45, 900

```

```

UNION ALL
SELECT 'Computer X', 'Electronics', 2025, 50, 950
UNION ALL
SELECT 'Computer X', 'Electronics', 2026, 60, 1100
UNION ALL
SELECT 'Chair', 'Furniture', 2024, 20, 600
UNION ALL
SELECT 'Chair', 'Furniture', 2025, 30, 750
UNION ALL
SELECT 'Table', 'Furniture', 2024, 10, 900
UNION ALL
SELECT 'Table', 'Furniture', 2026, 20, 950
),

```

```

Sales_revenue AS (
  SELECT *,
    price * volume AS revenue
  FROM Sales
),

```

```

Product_totals AS (
  SELECT
    product,
    category,
    SUM(revenue) AS total_revenue
  FROM Sales_revenue
  GROUP BY product, category
)

```

```

SELECT *,
  RANK() OVER (
    PARTITION BY category
    ORDER BY total_revenue DESC
  ) AS total_revenue_rank
FROM Product_totals;

```

exercise 10:

In the year of 2026, create per product the probability of buying it out:

- All year sales (by volume)
- The category sale (also by volume)

```

WITH Sales AS (
  SELECT 'Phone' AS product, 'Electronics' AS category, 2026 AS `year`, 20 AS
volume, 800 AS price
  UNION ALL
  SELECT 'Computer X', 'Electronics', 2026, 40, 400
  UNION ALL
  SELECT 'Chair', 'Furniture', 2026, 40, 230
  UNION ALL

```

```

SELECT 'Table', 'Furniture', 2026, 15, 180
UNION ALL
SELECT 'Couch', 'Furniture', 2026, 30, 780
),

Sales_volume AS (
  SELECT *,
    SUM(volume) OVER () AS total_volume,
    SUM(volume) OVER (
      PARTITION BY category
    ) AS total_volume_per_category
  FROM Sales
),

Sales_probability AS (
  SELECT *,
    volume / total_volume AS prop_volume,
    volume / total_volume_per_category AS prop_volume_per_category
  FROM Sales_volume
)

SELECT *
FROM Sales_probability;

```

exercise 11:

In the year of 2026, order the products by price and compute the cumulative probability of finding a product up to a product price.

```

WITH Sales AS (
  SELECT 'Phone' AS product, 'Electronics' AS category, 2026 AS `year`, 20 AS
  volume, 800 AS price
  UNION ALL
  SELECT 'Computer X', 'Electronics', 2026, 40, 400
  UNION ALL
  SELECT 'Chair', 'Furniture', 2026, 40, 230
  UNION ALL
  SELECT 'Table', 'Furniture', 2026, 15, 180
  UNION ALL
  SELECT 'Couch', 'Furniture', 2026, 30, 780
)

SELECT *,
  CUME_DIST() OVER (
    ORDER BY price
  ) AS cumulative_prop
FROM Sales;

```